

Frontend Technical Exercise — Bioprocess Medium Optimisation Dashboard

Context

In bioprocess development, scientists frequently compare a **reference medium formulation** against an **optimised formulation** generated through simulation or optimisation workflows.

Each medium contains multiple compounds (salts, nutrients, amino acids, sugars, etc.) with different concentration values and units.

Your task is to build a small but production-like visualisation module that allows scientists to:

- compare optimised vs reference concentrations,
- switch between absolute and relative visualisation modes,
- visually identify increases/decreases in formulation composition.

The goal of this exercise is to evaluate:

- Angular architecture and component design,
- data transformation and modelling,
- chart integration,
- UI clarity,
- testing approach,
- code readability and maintainability.

Technical Stack

- Angular 17+
- TypeScript
- SCSS
- Apache ECharts (`echarts` + `ngx-echarts`)

Expected Time

Approximately **3–4 hours**.

What You Need To Build

Build a reusable chart component that visualises the composition of a bioprocess medium.

The chart must compare:

- **Reference formulation**
- **Optimised formulation**

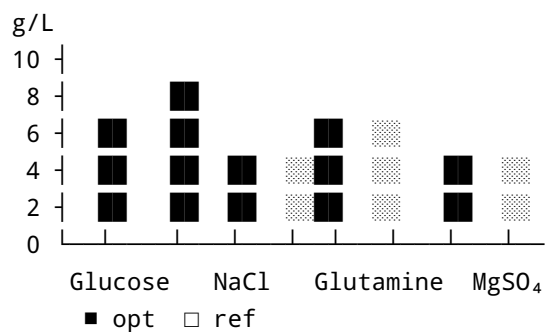
for every compound in the medium.

The user must be able to switch between:

1. **Absolute Mode**
2. Raw concentration values.
3. **Relative Mode**
4. Optimised value expressed relative to the reference.
5. Example:

```
relative = opt_value / ref_value
```

Example Visualisation



Dataset

```
export interface MediumComponent {
  name: string;
  unit: string;
  ref_value: number;
  opt_value: number;
}

export interface Medium {
  name: string;
  components: MediumComponent[];
}

export const MOCK_MEDIUM: Medium = {
  name: 'Basal Medium',
  components: [
    { name: 'Glucose', unit: 'g/L', ref_value: 4.0, opt_value: 4.8 },
    { name: 'NaCl', unit: 'g/L', ref_value: 8.0, opt_value: 7.2 },
    { name: 'Glutamine', unit: 'mmol/L', ref_value: 2.5, opt_value: 3.2 },
    { name: 'MgSO4', unit: 'g/L', ref_value: 0.5, opt_value: 0.65 },
  ],
};
```

Requirements

1. Data Transformation Layer

Create a pure transformation function:

```
buildChartData(medium, mode)
```

The responsibility of this function is to transform the domain data into chart-ready data.

Modes

Absolute Mode

- Use raw concentration values.
- Y-axis label:

Concentration

Relative Mode

Rules:

```
optimised = opt_value / ref_value  
reference = always 1.0
```

Additional behaviour:

- Y-axis label:

Relative to reference

- Add a dashed horizontal reference line at:

y = 1.0

This line represents the reference formulation baseline.

Visual Requirements

- Insert an invisible spacing/gap entry between compound groups.
- Each compound pair must share the same base colour.
- Reference bars should use a lighter shade.

Use this palette:

```
['#4A90D9', '#E07048', '#50B86C', '#9B6EC8']
```

2. Unit Tests

Write at least 3 unit tests covering:

1. Correct absolute values.
2. Correct relative calculations.
3. Presence of gap entries.

You may add additional tests if desired.

3. MediumChartComponent

Create a reusable Angular component.

Inputs

```
@Input() medium: Medium
@Input() mode: 'absolute' | 'relative'
```

Default mode:

```
'absolute'
```

Responsibilities

The component should:

- call the transformation function,
 - render the chart using ECharts,
 - reactively update when mode changes,
 - display the dashed reference line in relative mode,
 - show a clean tooltip containing:
 - compound name,
 - displayed value,
 - unit.
-

4. Page / Container

Create a simple page that:

- renders the chart using `MOCK_MEDIUM`,
- contains a two-button toggle:

```
[ Absolute ] [ Relative ]
```

- updates the chart dynamically.
-

Expected Deliverables

Please provide:

- Angular source code,
 - unit tests,
 - brief README with:
 - installation steps,
 - how to run,
 - assumptions or tradeoffs.
-

Evaluation Criteria

Your solution will be evaluated based on:

Area	What We Look For
Architecture	Clear separation of concerns
TypeScript	Strong typing and clean modelling
Angular	Component structure and reactivity
Visualisation	Correct and readable chart behaviour
Testing	Meaningful unit coverage
Code Quality	Readability, maintainability, naming
UX	Clarity and usability

Bonus (Optional)

If time allows, consider adding:

- responsive layout,
 - loading/empty states,
 - animation transitions,
 - legend improvements,
 - accessibility improvements,
 - support for many compounds,
 - support for grouped compounds/categories.
-

Notes

This exercise is intentionally focused on:

- data transformation,
- frontend architecture,
- charting logic,
- component reusability.

You are not expected to build a backend or implement advanced styling.

Clean, maintainable and well-structured code is preferred over over-engineering.